

Post Processing UIs



Renan Araujo

GDE, Bluefire, Superlist

renan.gg

I am tired

(But that is my fault)

“The Post Processing API”

Flutter & Friends

September 1st, free to watch on YouTube

“Flutter, Flame & Fragment shaders”

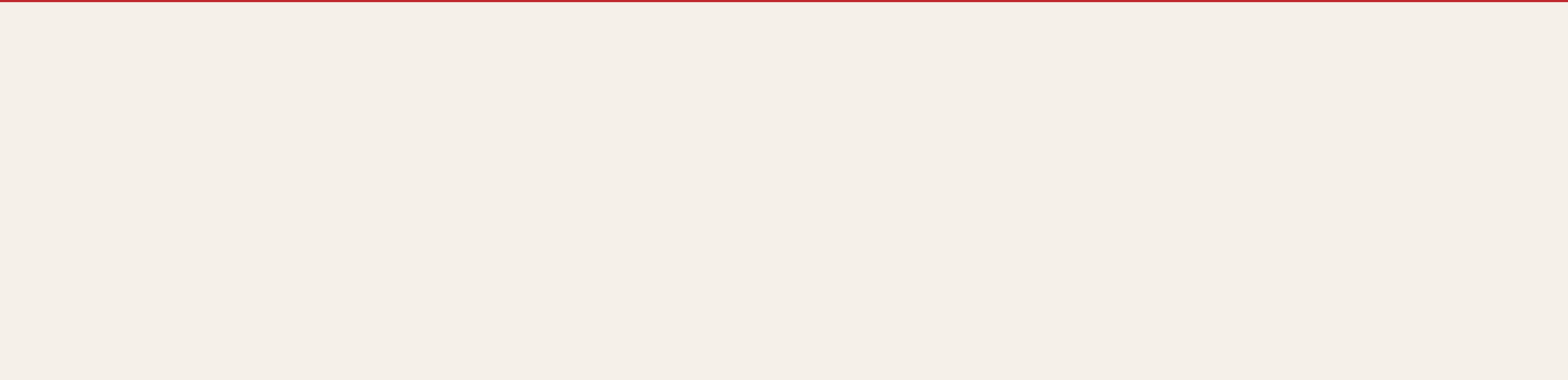
Fluttercon

September 24th, free to watch on YouTube

“Post Pocessing Uls”

Flutter Netherlands

Today, now... Duh



Flame

Fragment shaders

Flutter

Flame **Post-processing** API



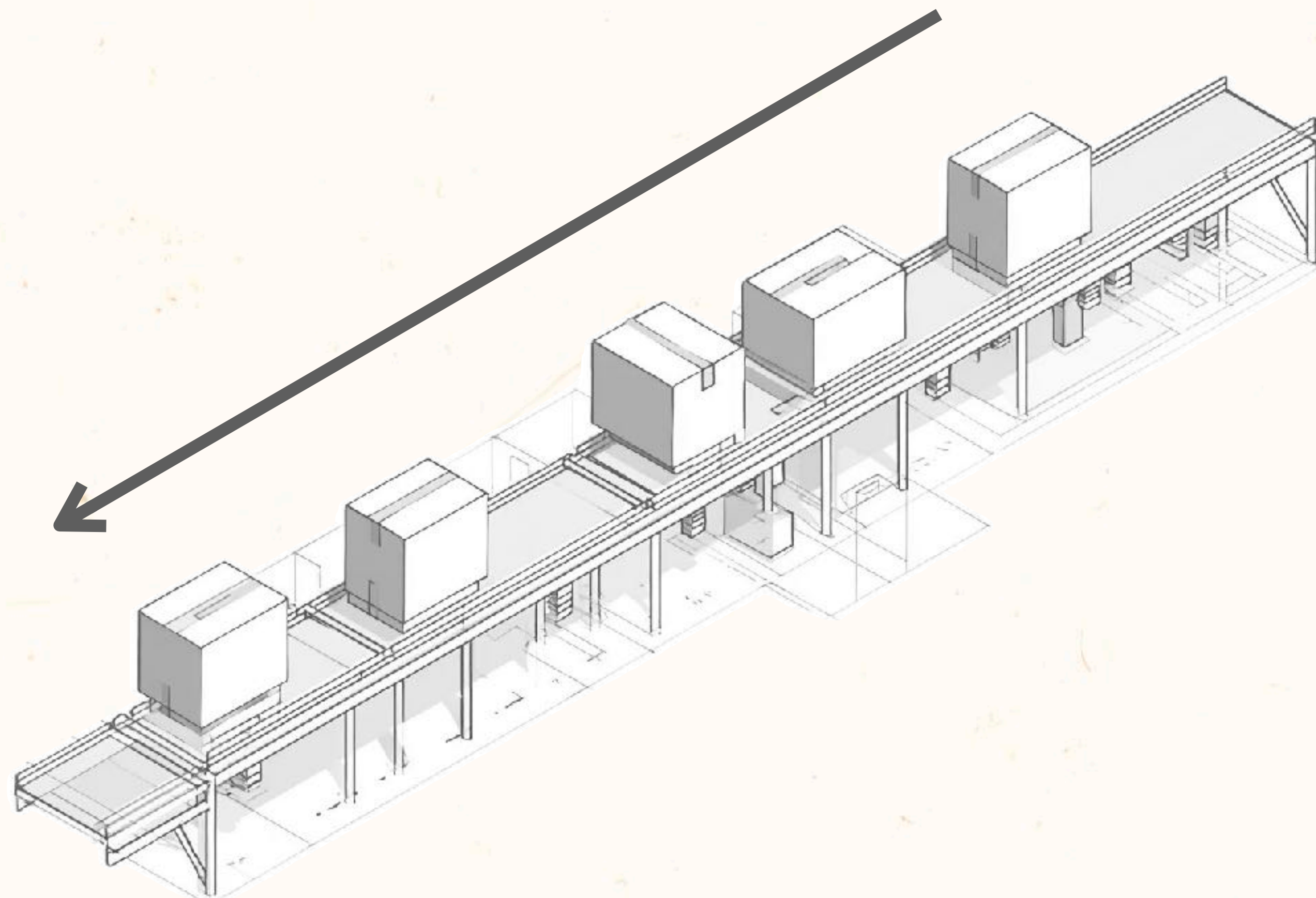
Flutter, Flame & Fragment shaders



Post Processing UIs

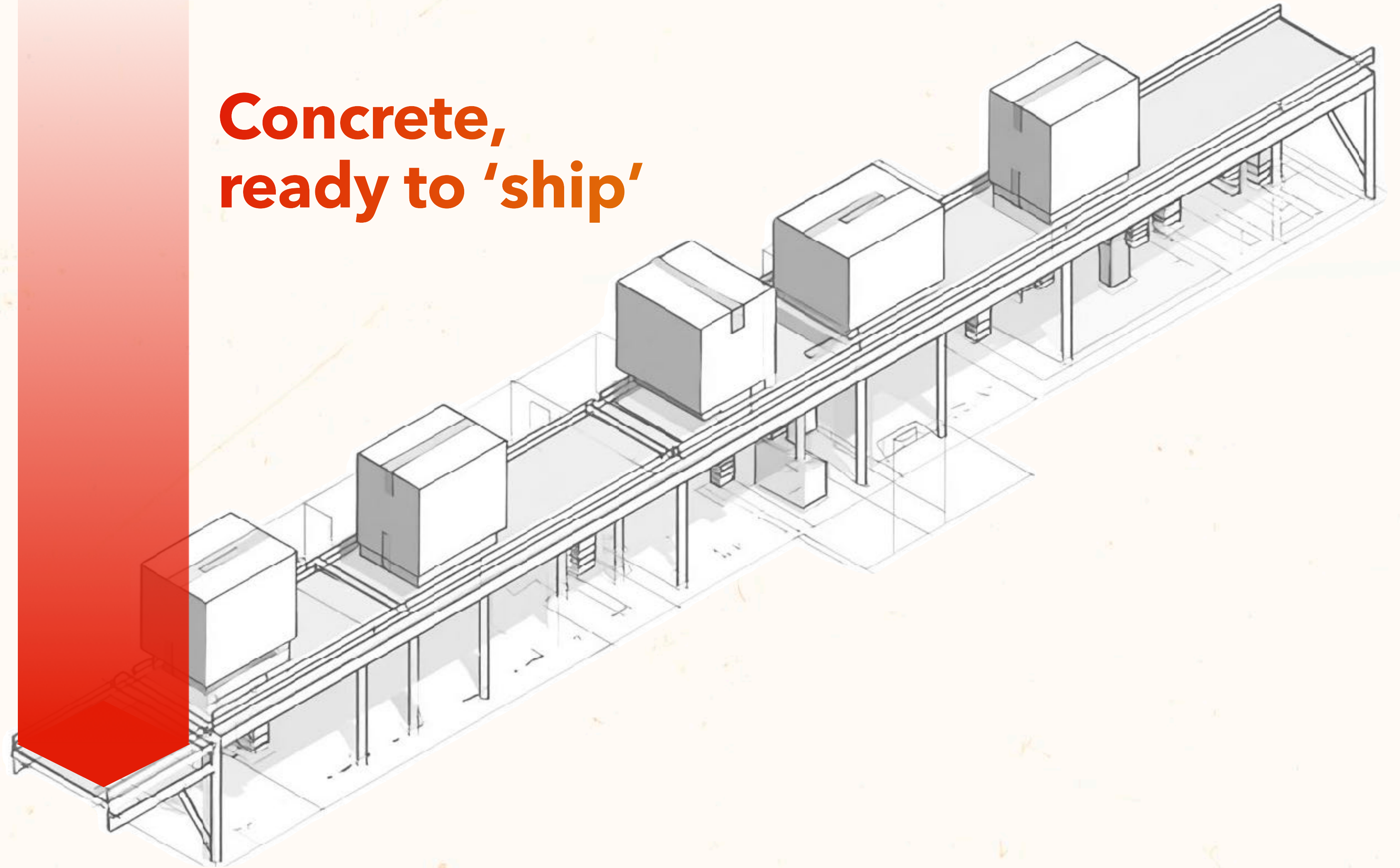
Chapter 1

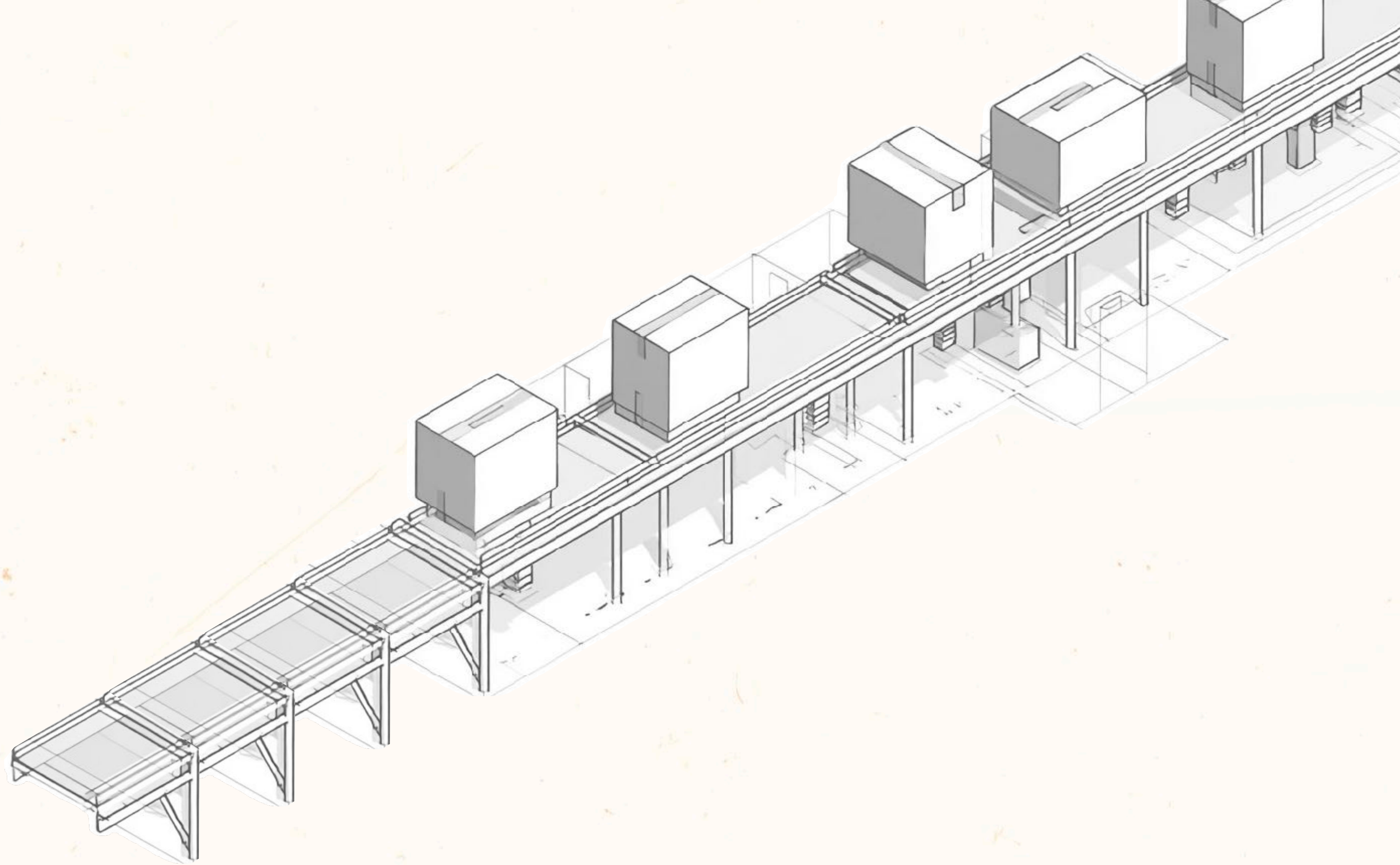
Post processing?



Abstract, raw

**Concrete,
ready to 'ship'**





The Flutter Factory of Pixels

The Flutter Factory of Pixels

```
class MyWidget extends StatelessWidget {  
  // ...  
  Widget build(BuildContext context) {  
    return Center(  
      child: ColoredBox(  
        color: Colors.amber,  
        child: SizedBox(  
          width: 100,  
          height: 100,  
        ),  
      ),  
    );  
  }  
}
```

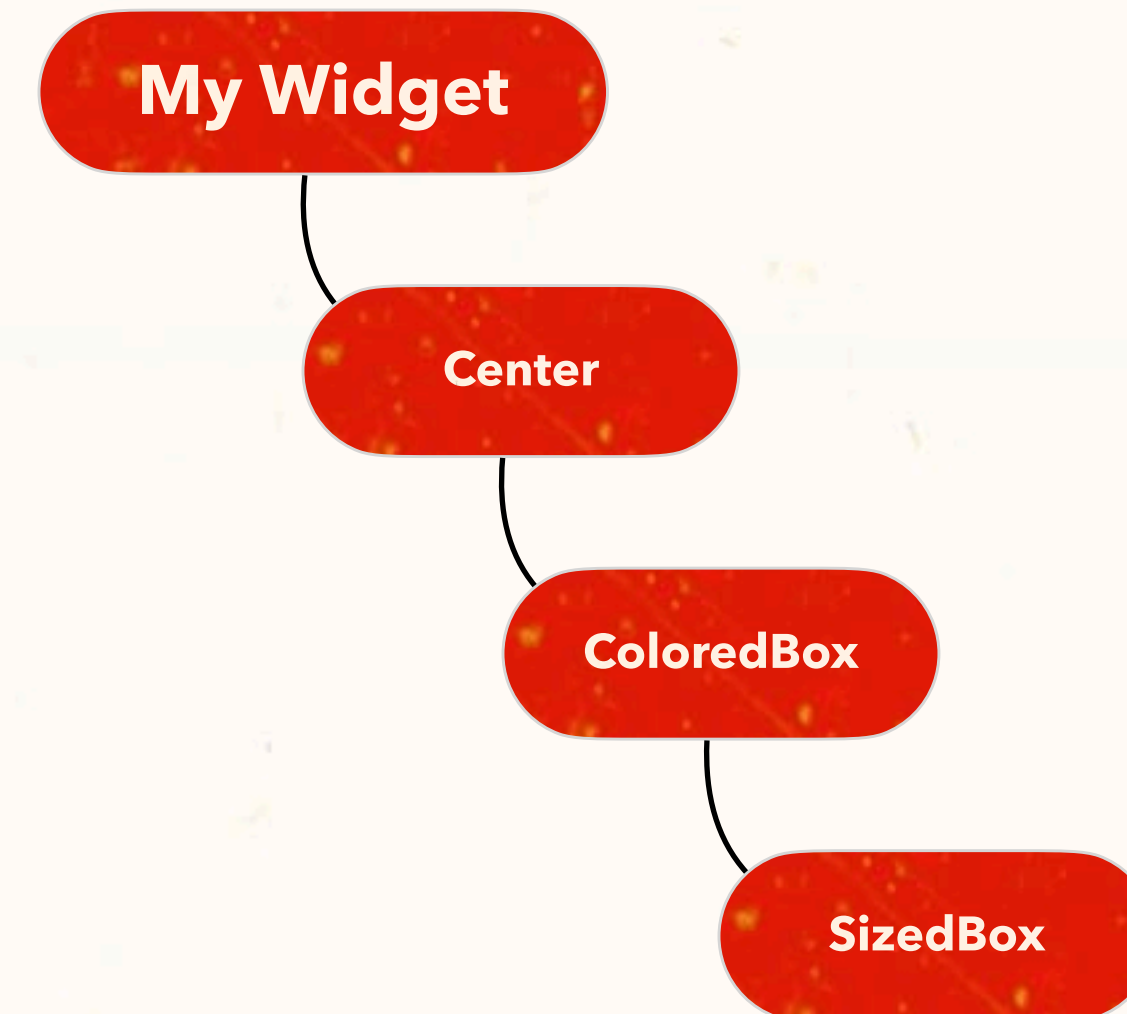
The Flutter Factory of Pixels

```
setState(() {  
  // ..  
});
```

```
class MyWidget extends StatelessWidget {  
  // ...  
  Widget build(BuildContext context) {  
    return Center(  
      child: ColoredBox(  
        color: Colors.amber,  
        child: SizedBox(  
          width: 100,  
          height: 100,  
        ),  
      ),  
    );  
  }  
}
```

The Flutter Factory of Pixels

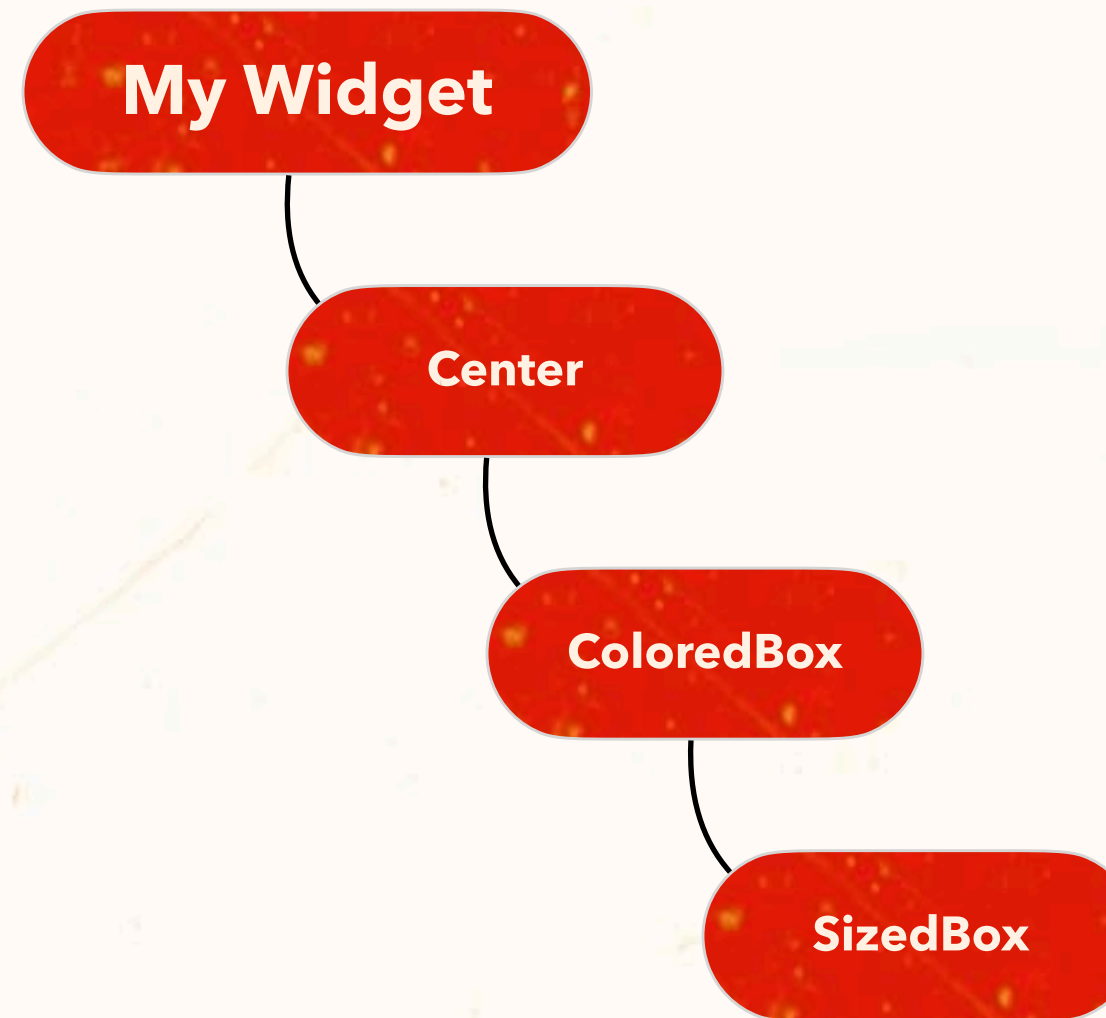
```
class MyWidget extends StatelessWidget {  
  // ...  
  Widget build(BuildContext context) {  
    return Center(  
      child: ColoredBox(  
        color: Colors.amber,  
        child: SizedBox(  
          width: 100,  
          height: 100,  
        ),  
      ),  
    );  
  }  
}
```



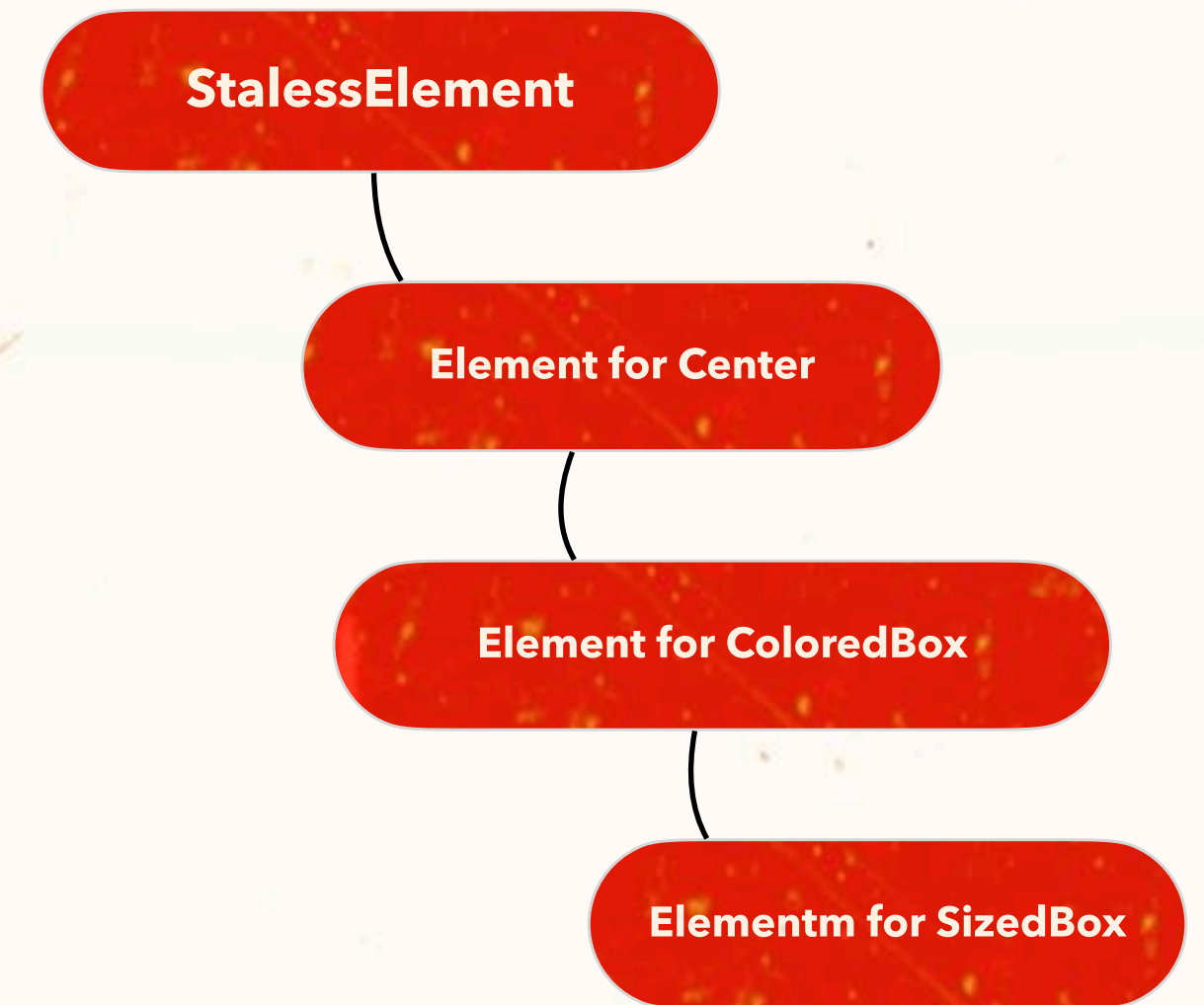
Widget tree

The Flutter Factory of Pixels

```
class MyWidget extends StatelessWidget {  
  // ...  
  Widget build(BuildContext context) {  
    return Center(  
      child: ColoredBox(  
        color: Colors.amber,  
        child: SizedBox(  
          width: 100,  
          height: 100,  
        ),  
      ),  
    );  
  }  
}
```



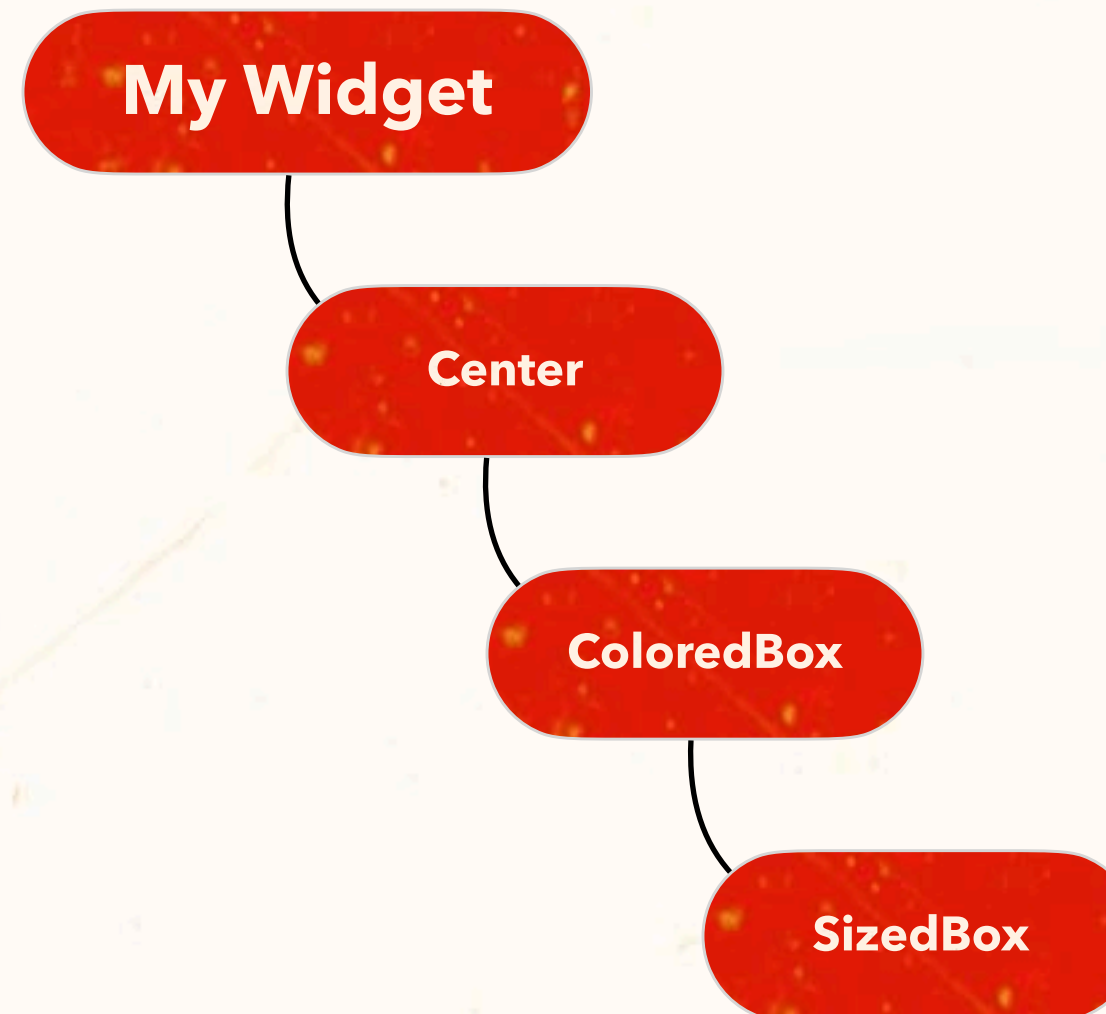
Widget tree



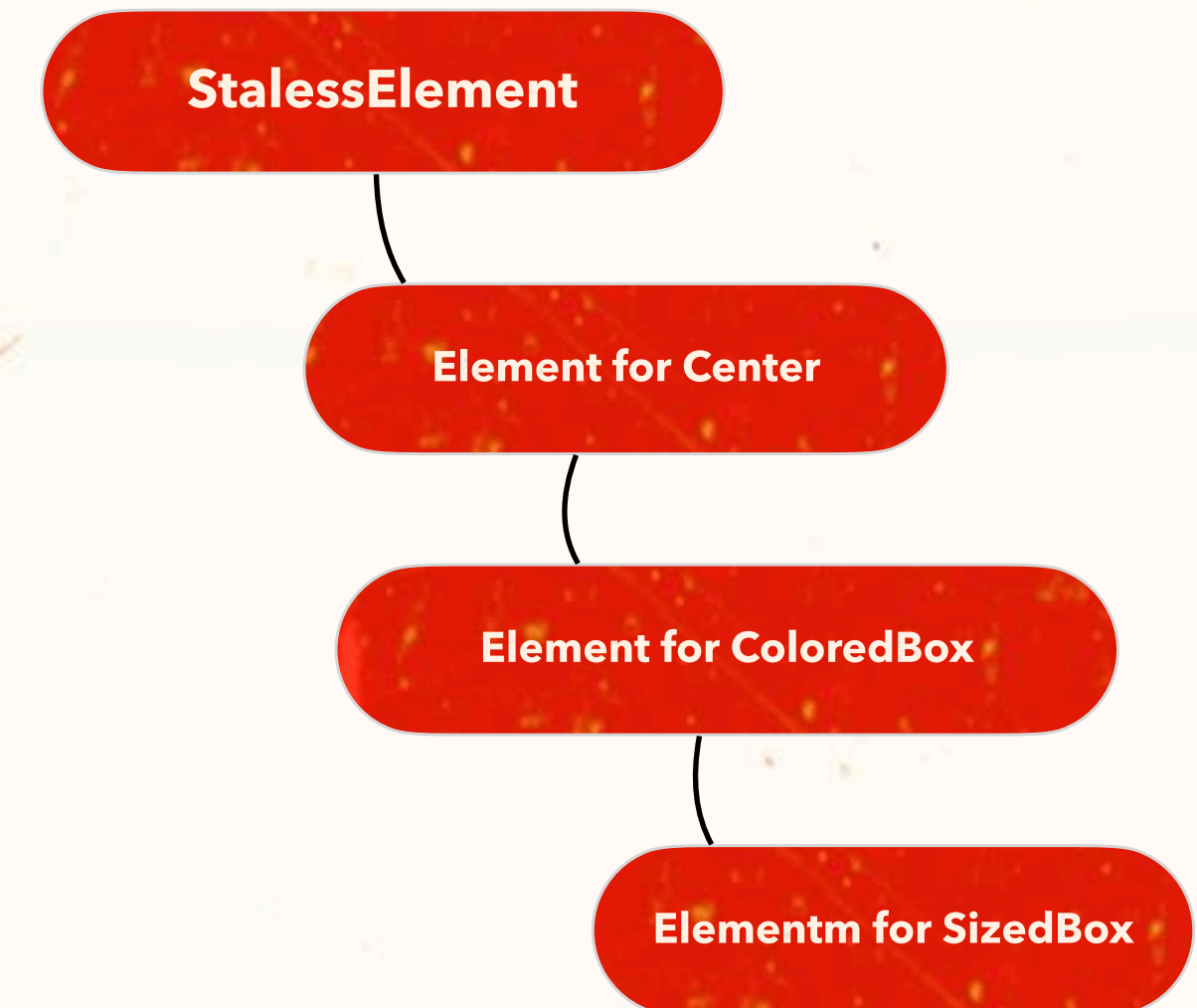
Element tree

The Flutter Factory of Pixels

```
class MyWidget extends StatelessWidget {  
  // ...  
  Widget build(BuildContext context) {  
    return Center(  
      child: Opacity(  
        opacity: 0.8,  
        child: SizedBox(  
          width: 100,  
          height: 100,  
        ),  
      ),  
    );  
  }  
}
```



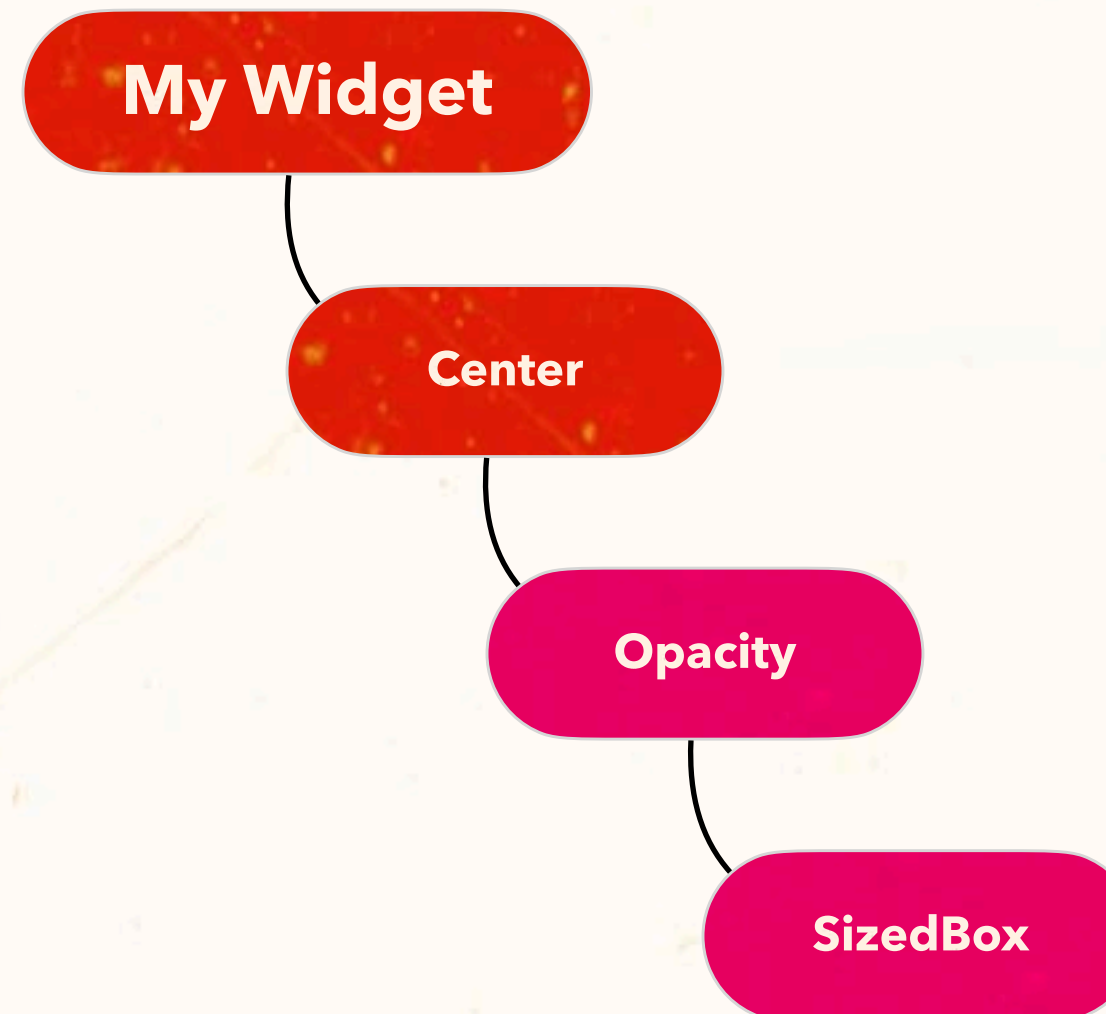
Widget tree



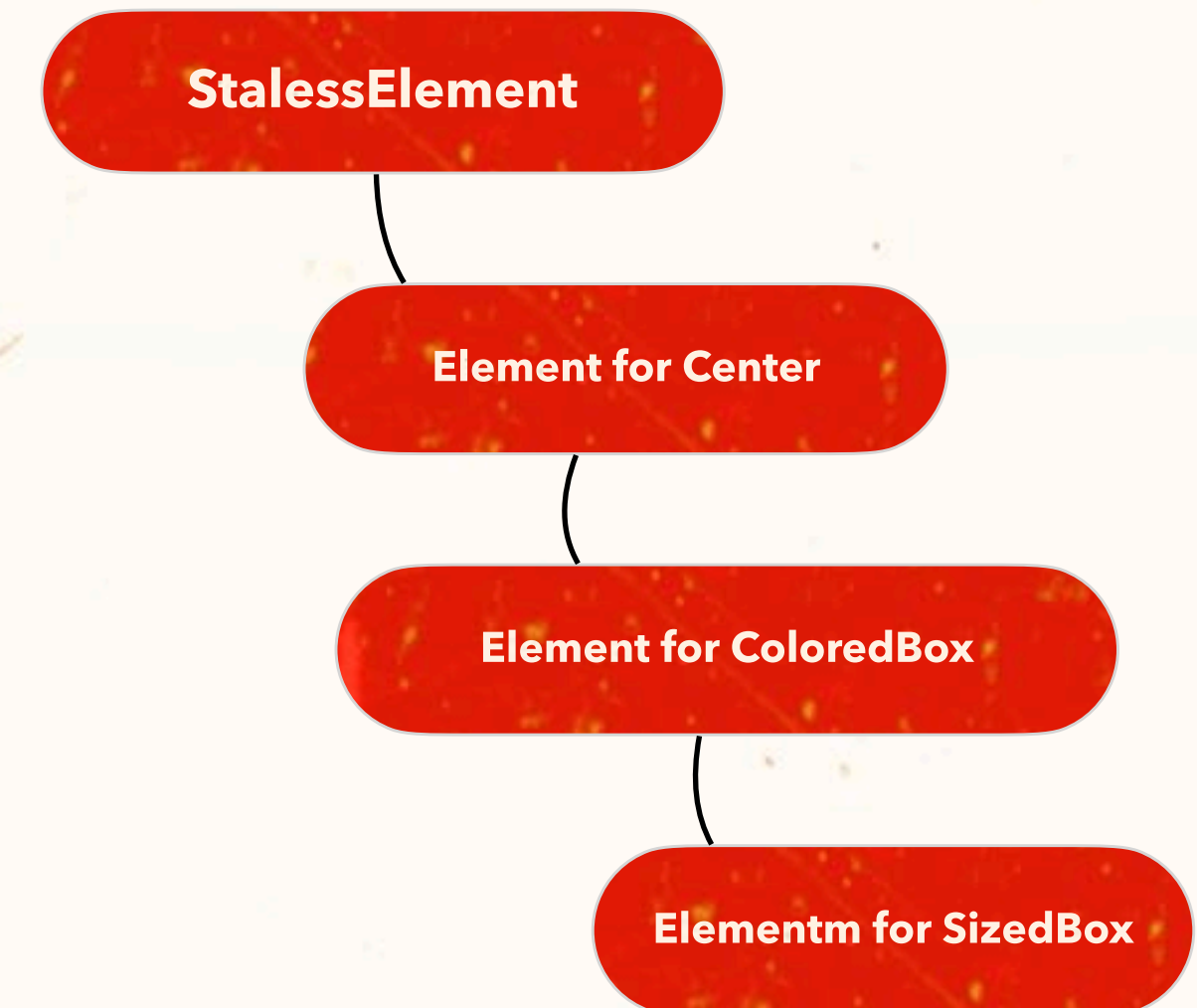
Element tree

The Flutter Factory of Pixels

```
class MyWidget extends StatelessWidget {  
  // ...  
  Widget build(BuildContext context) {  
    return Center(  
      child: Opacity(  
        opacity: 0.8,  
        child: SizedBox(  
          width: 100,  
          height: 100,  
        ),  
      ),  
    );  
  }  
}
```



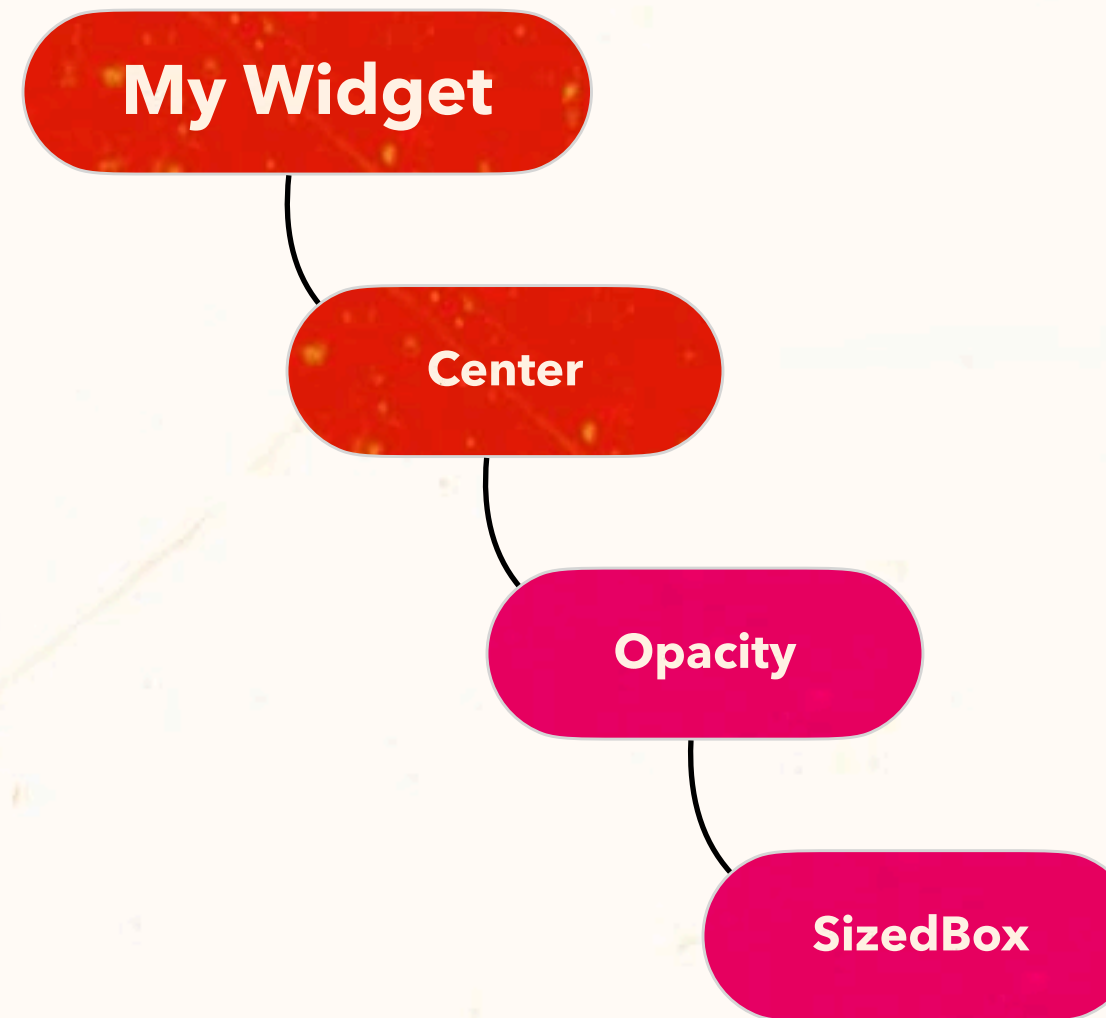
Widget tree



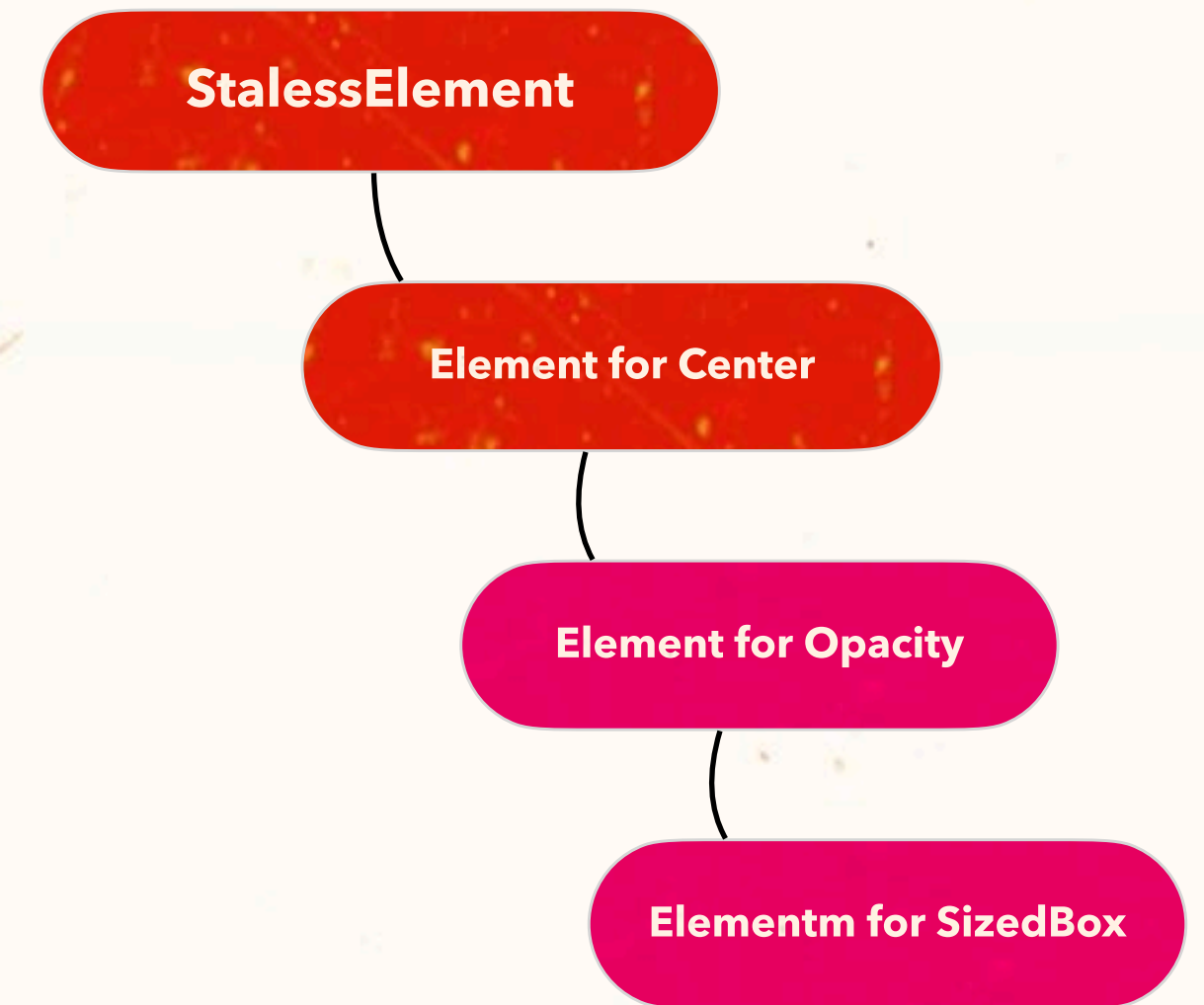
Element tree

The Flutter Factory of Pixels

```
class MyWidget extends StatelessWidget {  
  // ...  
  Widget build(BuildContext context) {  
    return Center(  
      child: Opacity(  
        opacity: 0.8,  
        child: SizedBox(  
          width: 100,  
          height: 100,  
        ),  
      ),  
    );  
  }  
}
```



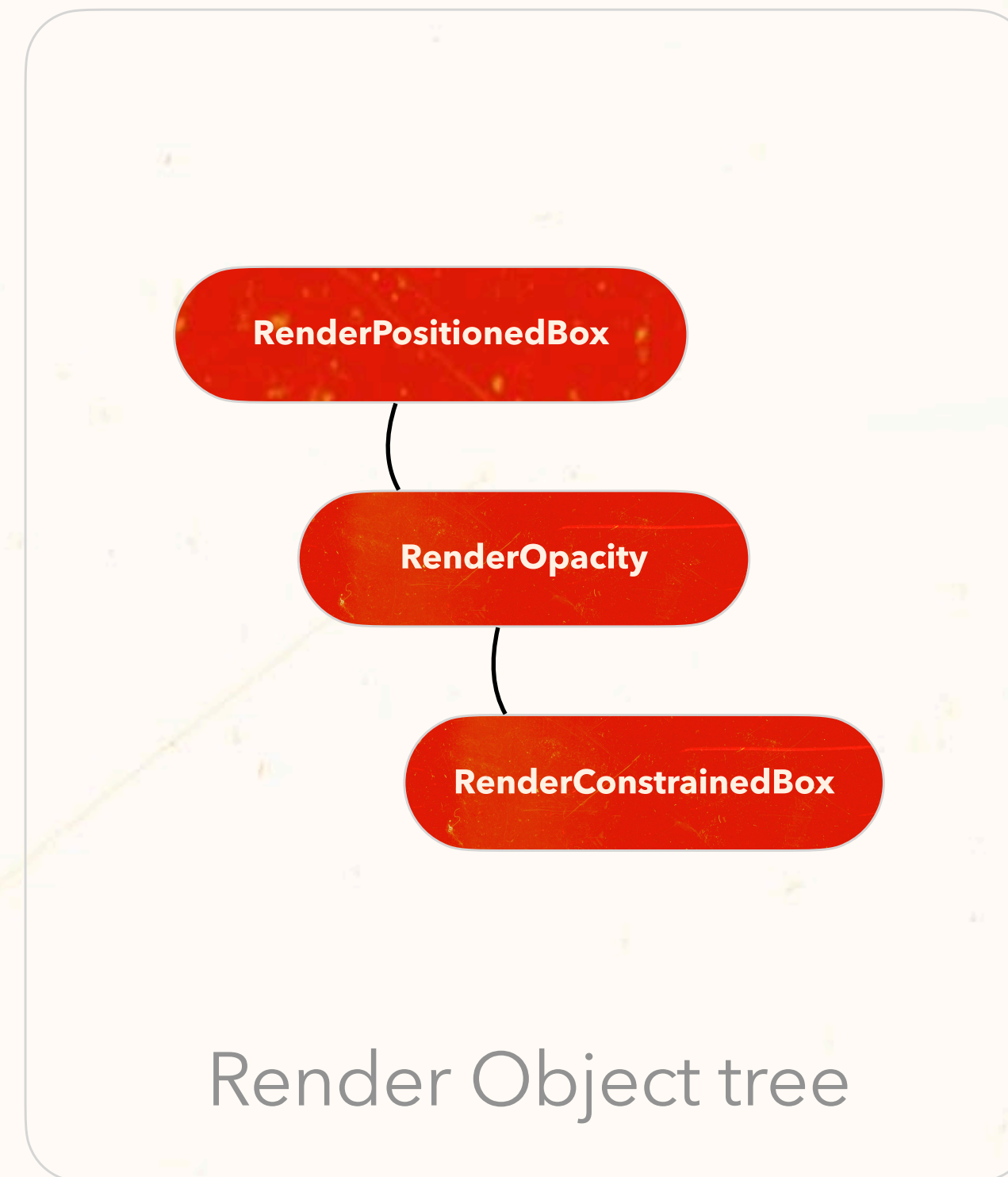
Widget tree



Element tree

Build

The Flutter Factory of Pixels



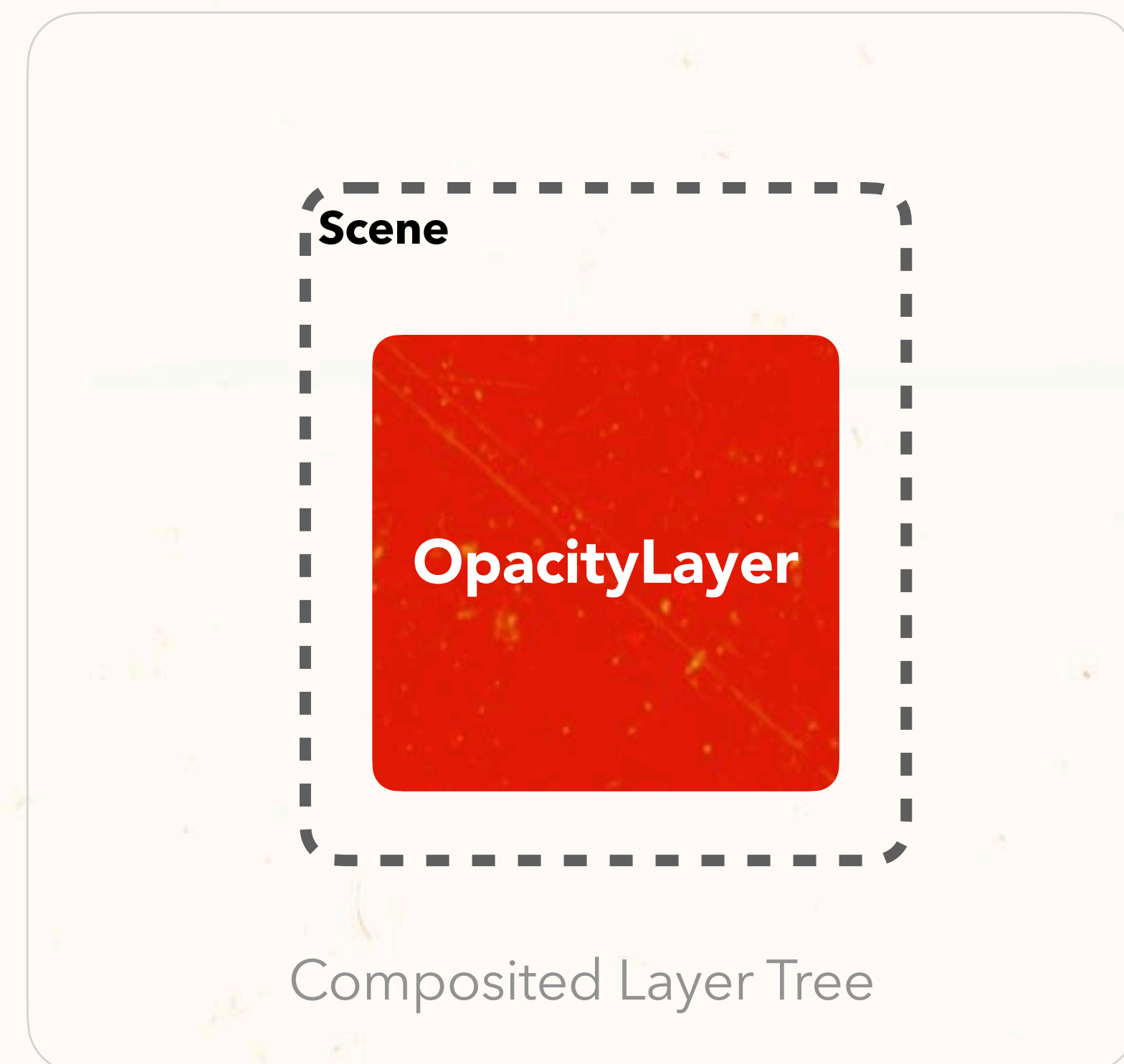
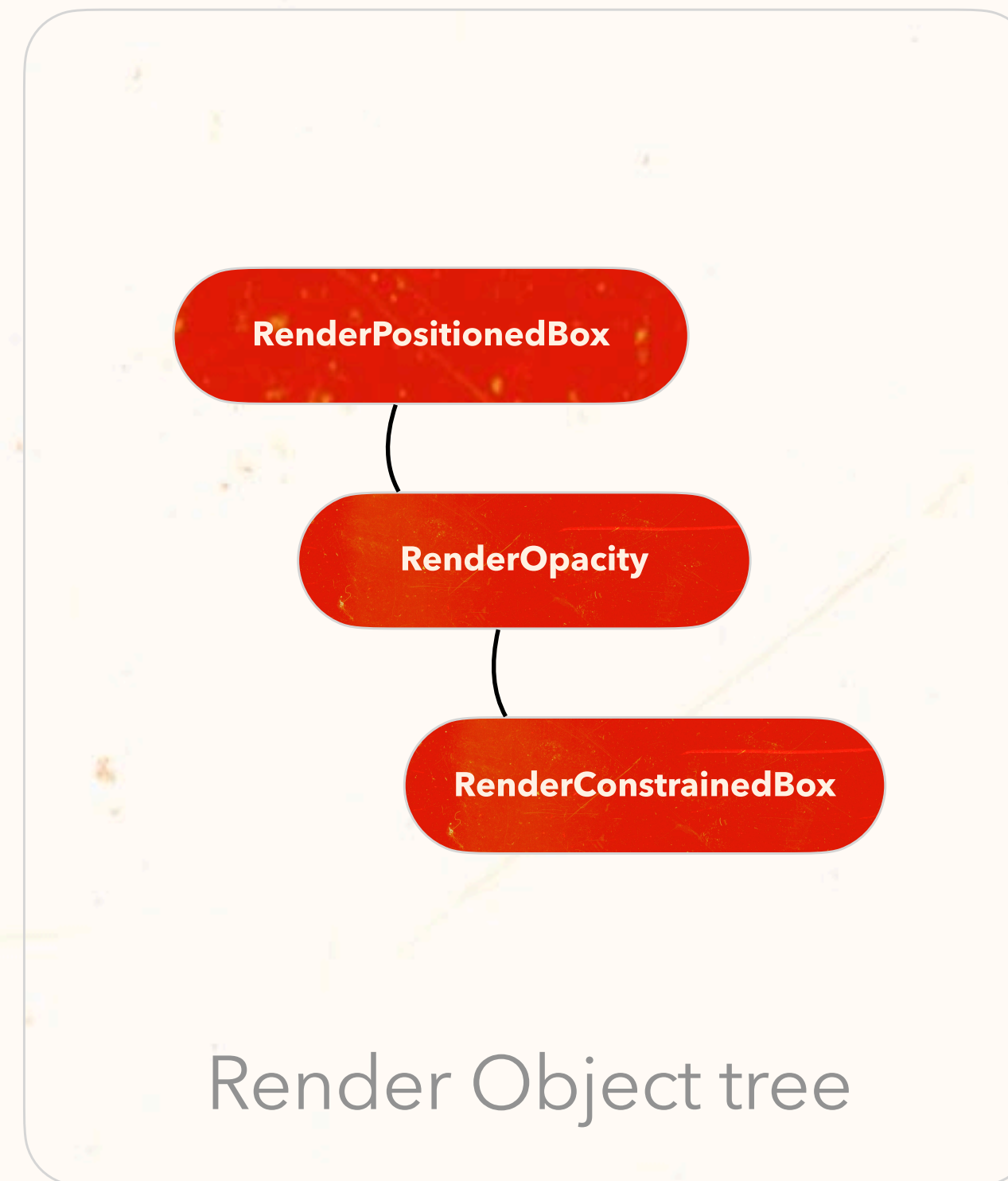
Build

Layout

Paint

Composition

The Flutter Factory of Pixels



Build

Layout

Paint

Composition

Examples of composited layers

Opacity

OpacityLayer

ImageFiltered

ImageFilterLayer

BackdropFilter

BackdropFilterLayer

Transform

TransformLayer

ShaderMask

ShaderMaskLayer

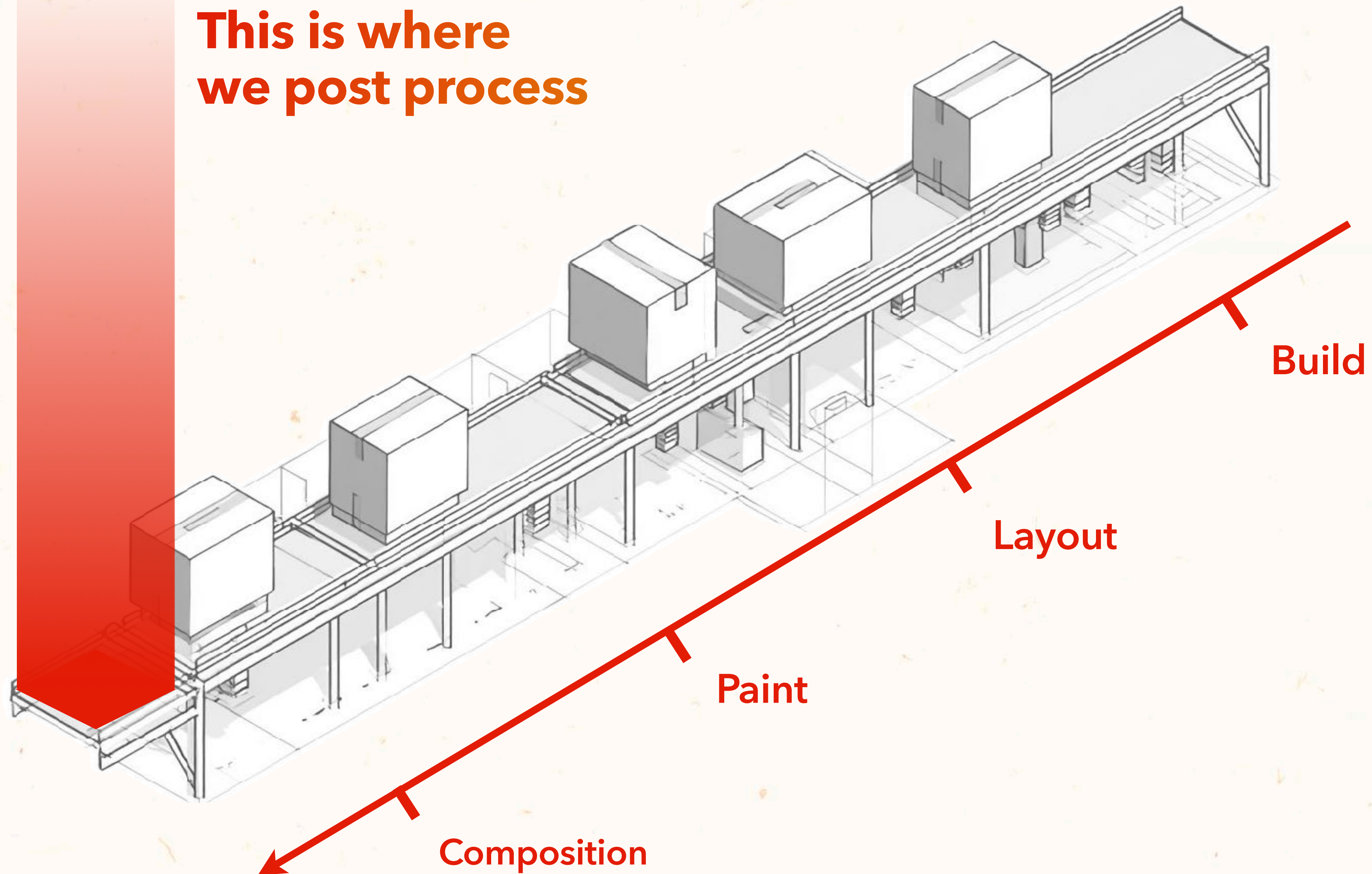
ClipRect

ClipRectLayer

...

Why?

**This is where
we post process**



Chapter 2

Shader?

```
#include <flutter/runtime_effect.glsl>

uniform vec2 uSize;

out vec4 fragColor;

void main() {
    vec2 pos = FlutterFragCoord().xy;
    vec2 uv = pos / uSize;

    float red = uv.x;
    float green = uv.y;
    float blue = 1.0 - red;

    fragColor = vec4(red, green, blue, 1.0);
}
```

```
#include <flutter/runtime_effect.glsl>

uniform vec2 uSize;

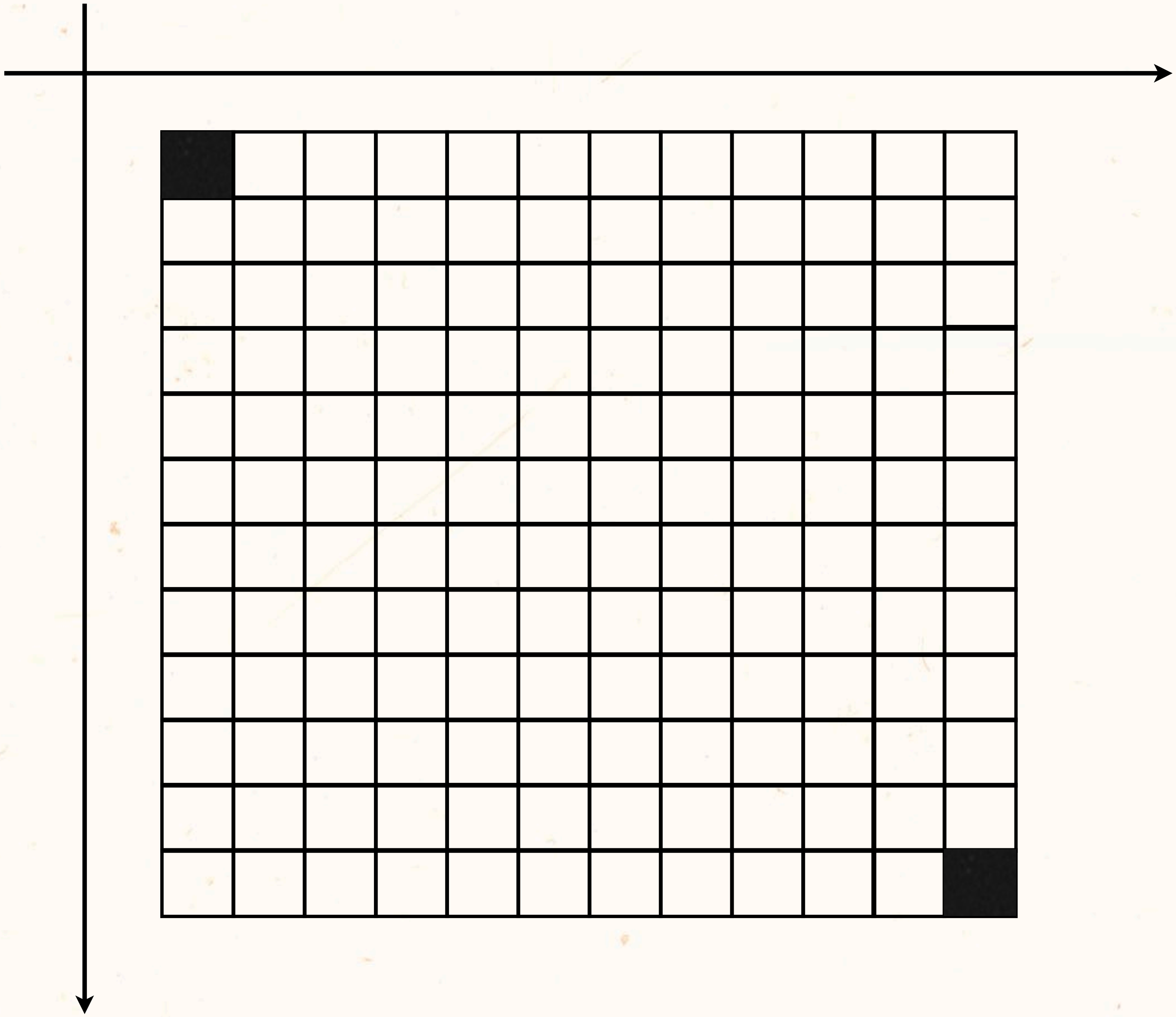
out vec4 fragColor;

void main() {
    vec2 pos = FlutterFragCoord().xy;
    vec2 uv = pos / uSize;

    float red = uv.x;
    float green = uv.y;
    float blue = 1.0 - red;

    fragColor = vec4(red, green, blue, 1.0);
}
```

**What color
should a pixel
have?**



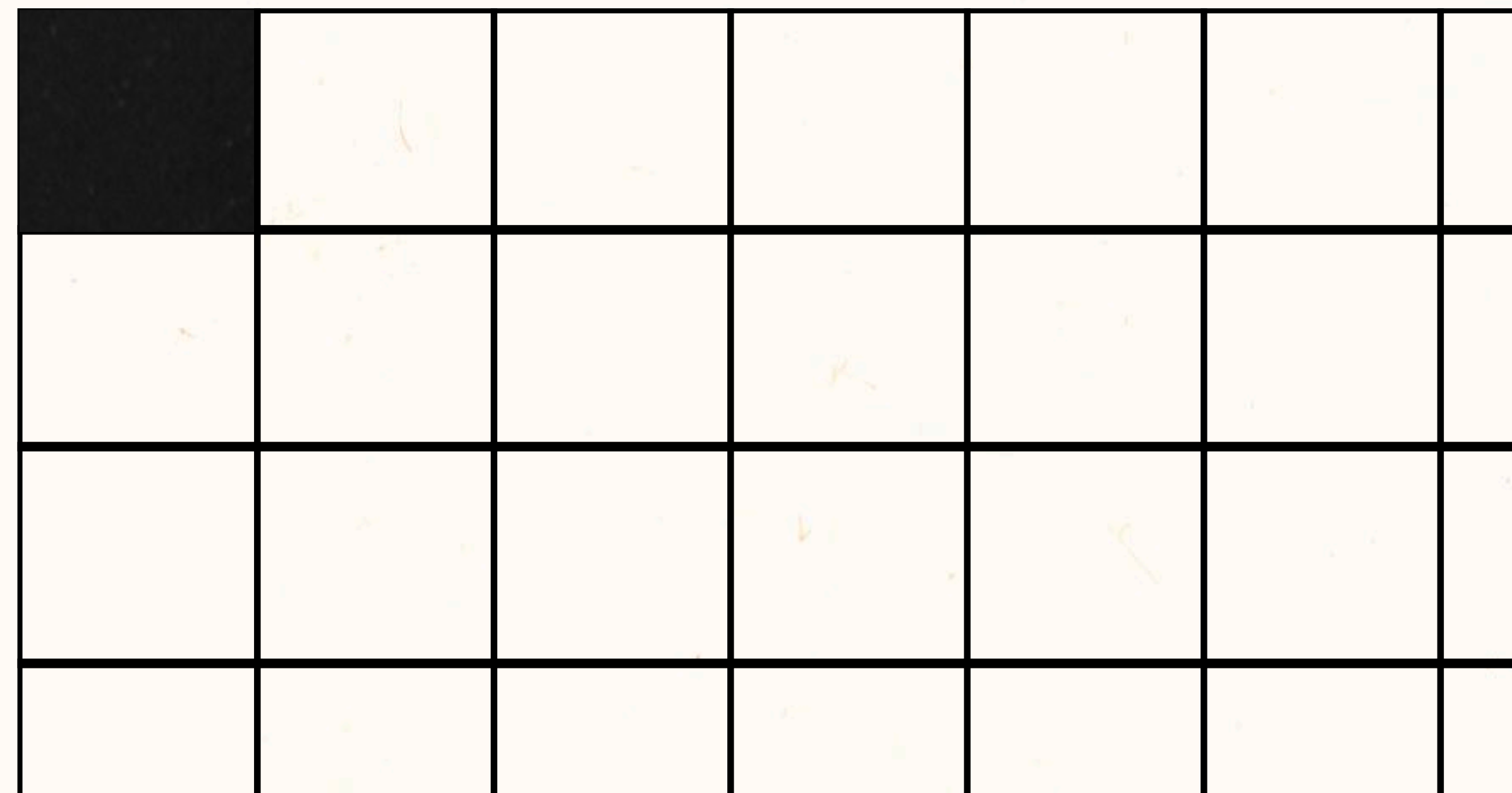
```
vec2 uv = vec2(0.0, 0.0);
```

```
float red = uv.x;
```

```
float green = uv.y;
```

```
float blue = 1.0 - red;
```

```
fragColor = vec4(red, green, blue, 1.0);
```



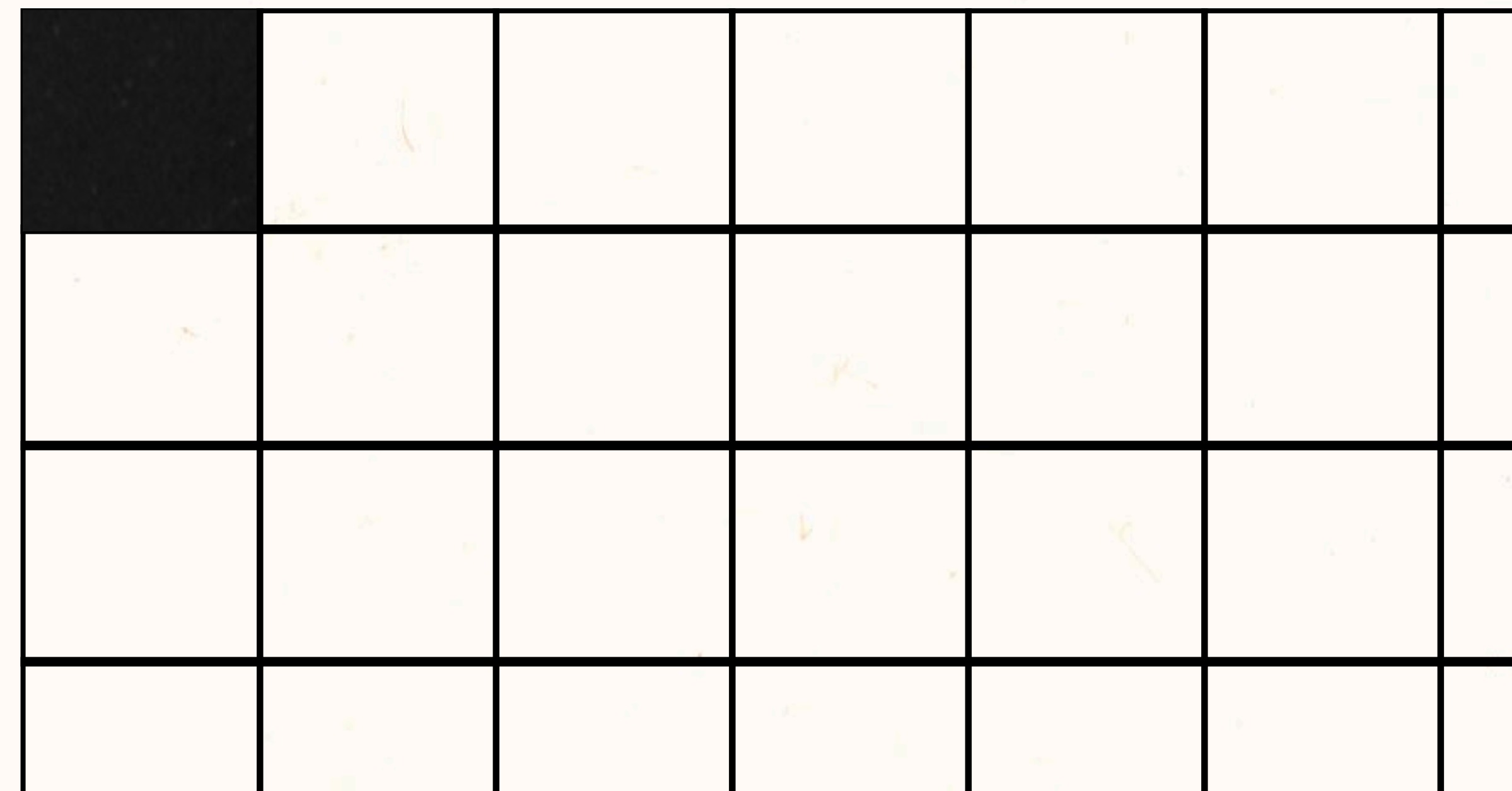
```
vec2 uv = vec2(0.0, 0.0);
```

```
float red = 0.0;
```

```
float green = 0.0;
```

```
float blue = 1.0 - 0.0;
```

```
fragColor = vec4(red, green, blue, 1.0);
```



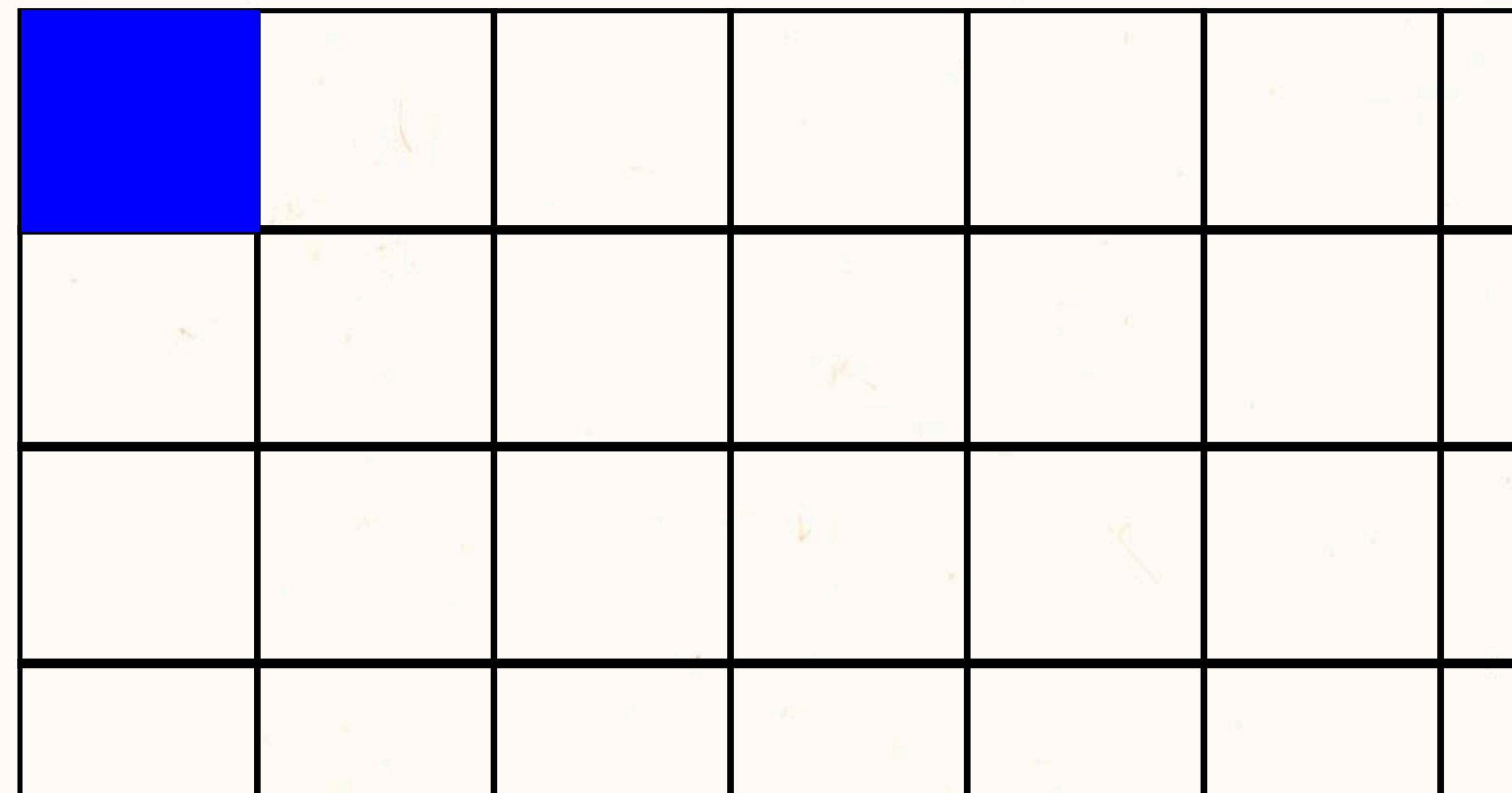
```
vec2 uv = vec2(0.0, 0.0);
```

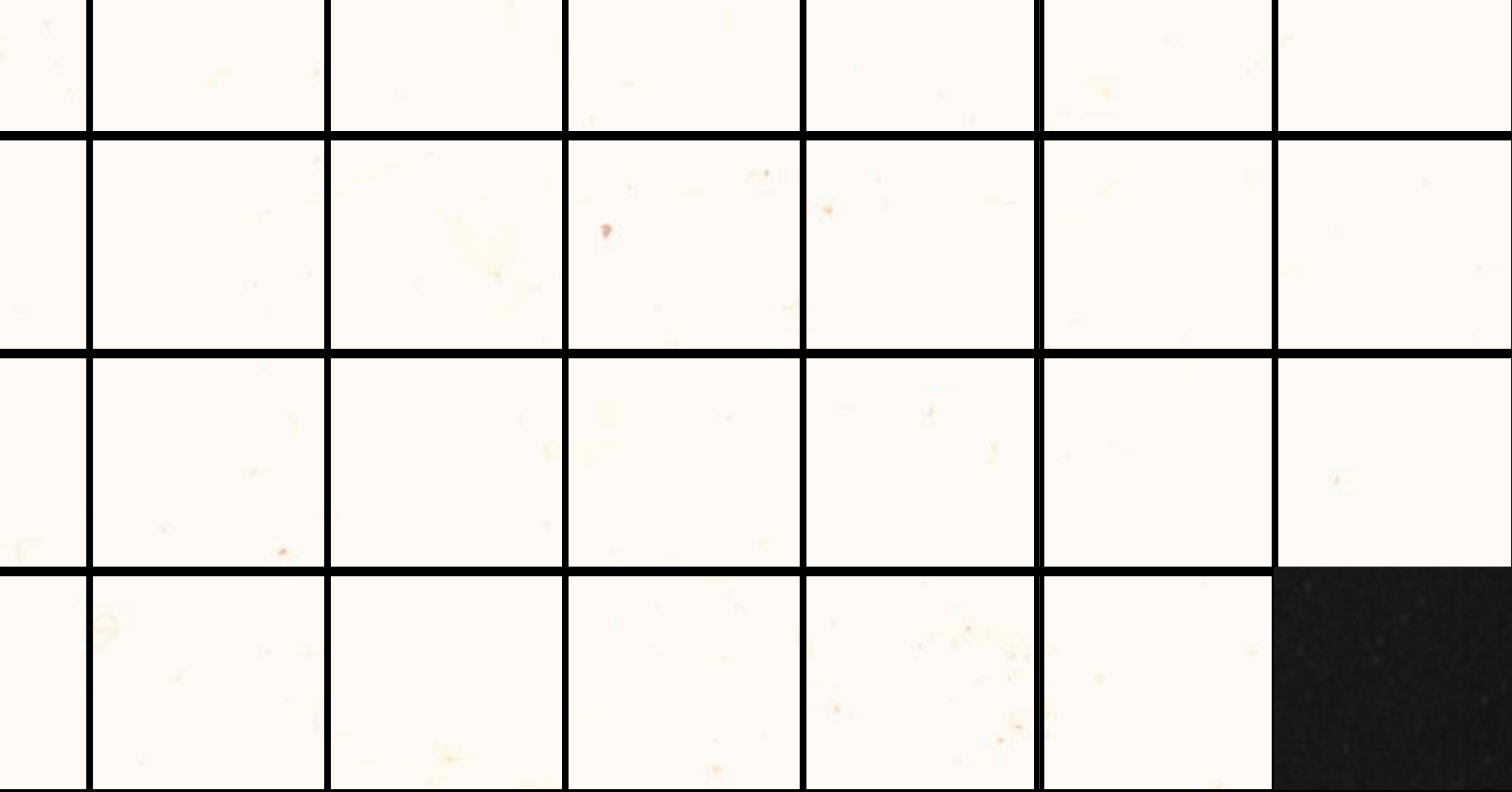
```
float red = 0.0;
```

```
float green = 0.0;
```

```
float blue = 1.0 - 0.0;
```

```
fragColor = vec4(red, green, blue, 1.0);
```





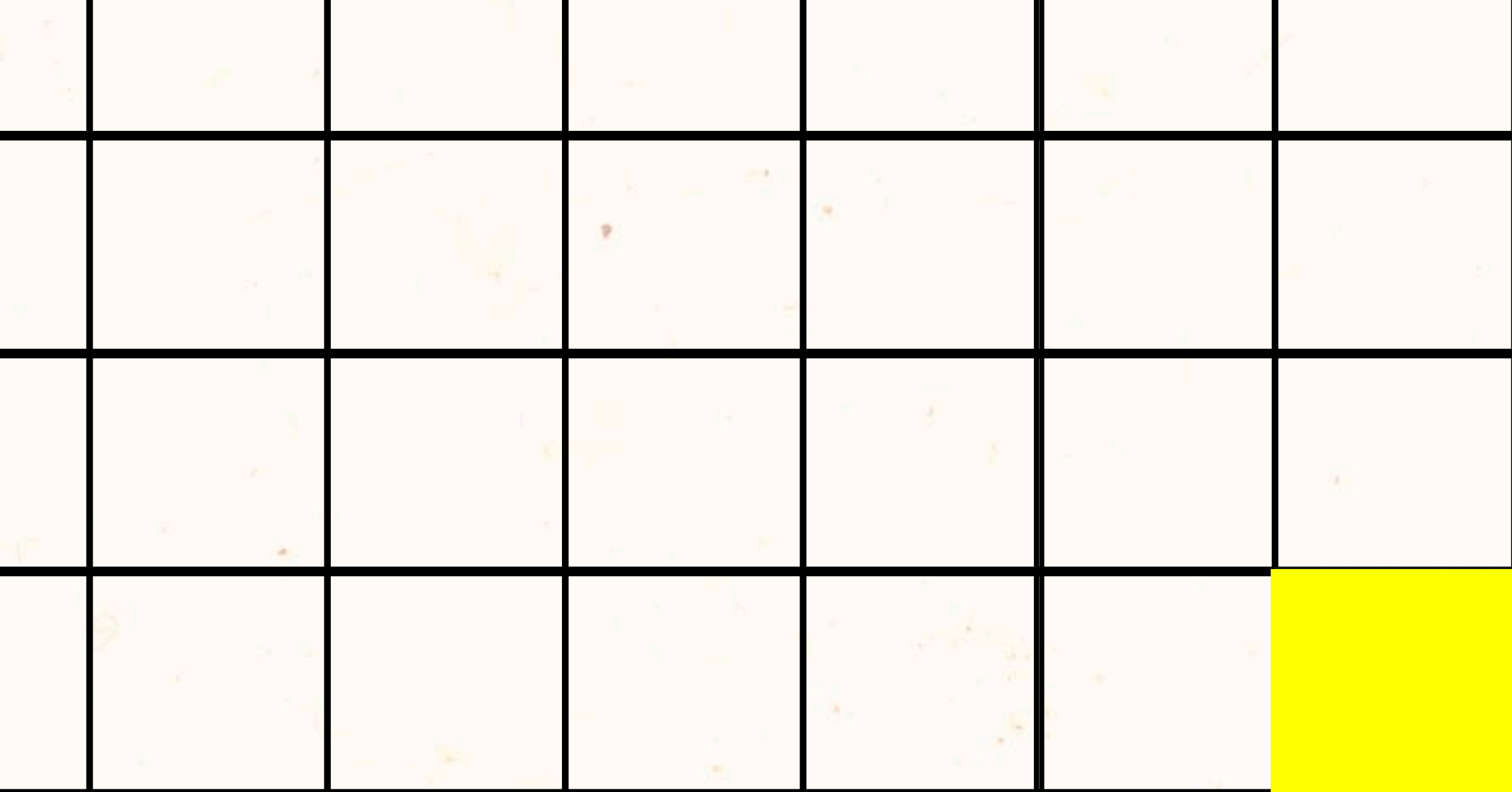
```
vec2 uv = vec2(1.0, 1.0);
```

```
float red = uv.x;
```

```
float green = uv.y;
```

```
float blue = 1.0 - red;
```

```
fragColor = vec4(red, green, blue, 1.0);
```



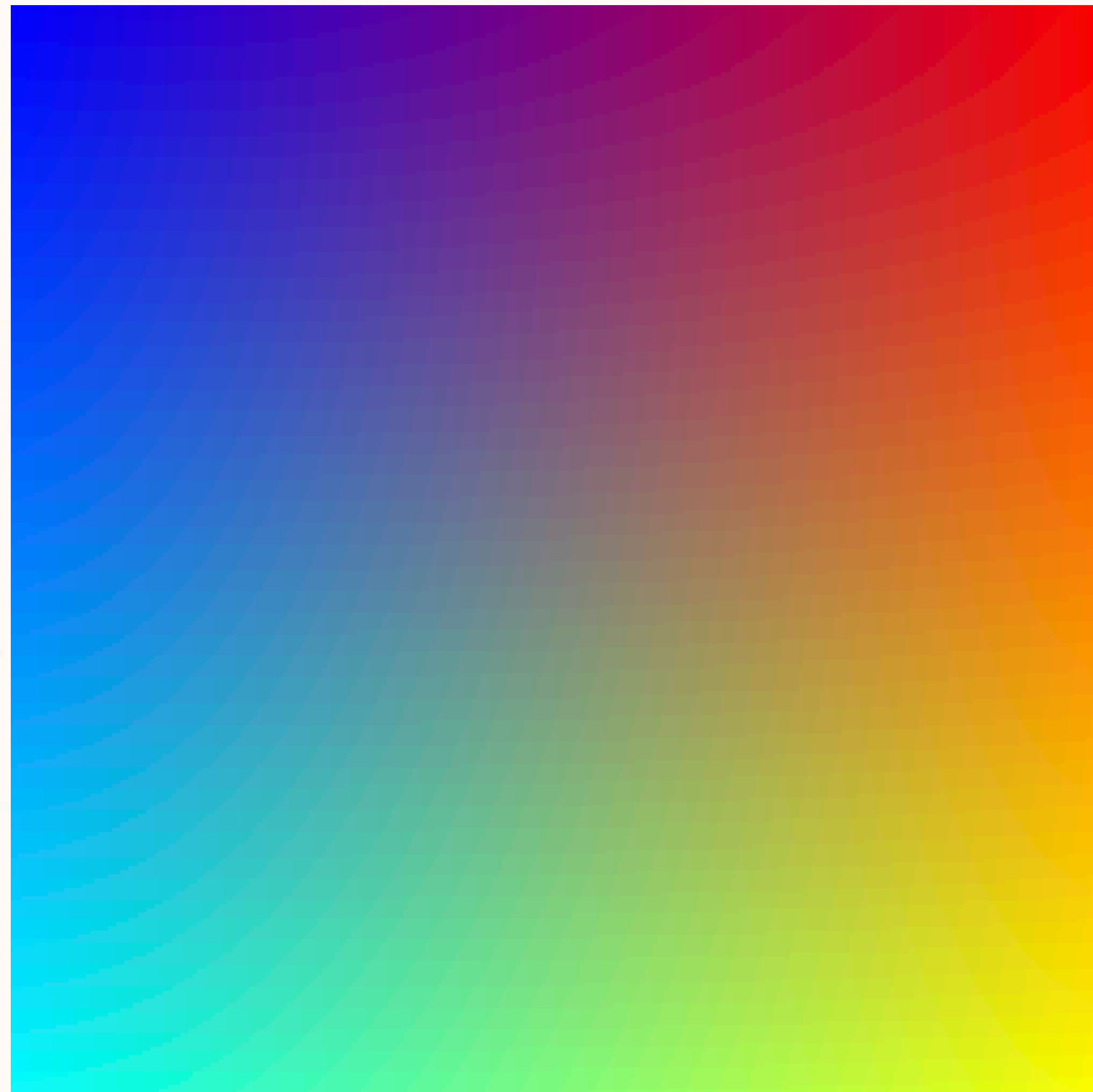
```
vec2 uv = vec2(1.0, 1.0);
```

```
float red = uv.x;
```

```
float green = uv.y;
```

```
float blue = 1.0 - red;
```

```
fragColor = vec4(red, green, blue, 1.0);
```



Inputs

```
uniform float uScale;  
uniform vec2 uSize;  
uniform vec4 uColor;  
uniform mat4 uTransform;  
uniform sampler2D uTexture;
```

- **Numeric**
- **Texture**

Chapter 3

Post processing Flutter Rendered Surfaces

flutter_shaders helper

```
Widget build(BuildContext context) {  
  return AnimatedSampler(  
    (ui.Image image, Size size, Canvas canvas) {  
      shader  
        ..setFloat(0, size.width)  
        ..setFloat(1, size.height)  
        ..setImageSampler(0, image);  
      canvas.drawRect(  
        Offset.zero & size,  
        Paint()..shader = shader,  
      );  
    },  
    child: widget.child,  
  );  
}
```

Special Composited Layer

```
final sb = SceneBuilder();
addChildToScene(sb);
Image image = sb.build().toImageSync();

final pictureRecorder = PictureRecorder();
final canvas = Canvas(pictureRecorder);

// ..

shader.setImageSampler(0, image);
canvas.drawRect(
    Offset.zero & size,
    Paint()..shader = shader,
);
```

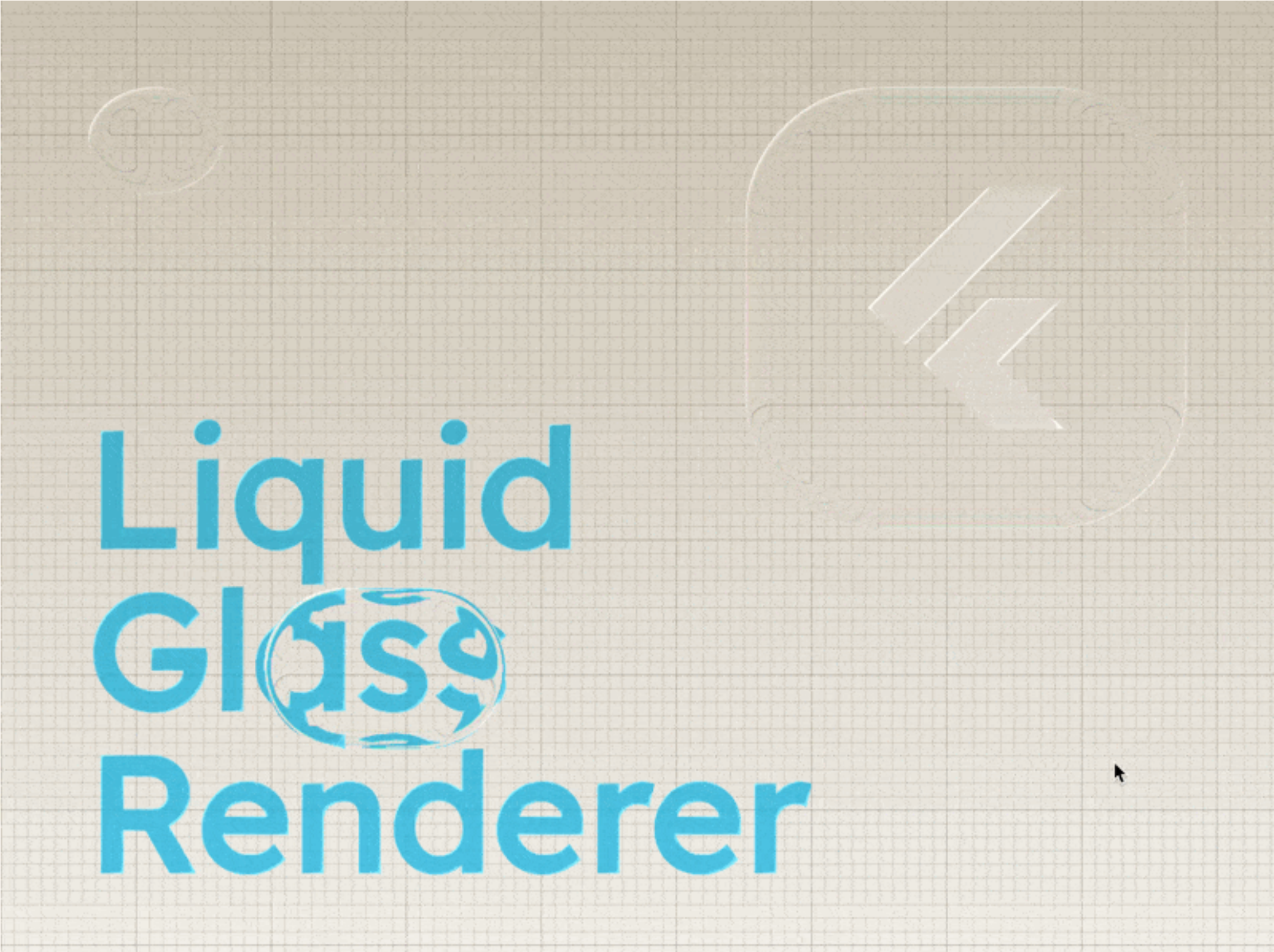
Chapter 4

Inspecting Liquido

22:26

22:26

22:26



Liquid
Glass
Renderer

**Liquid Glass
Renderer**

NAN

renan.gg